Experiment 3

AIM : A Random Forest is an "Ensemble" method. Instead of one tree, it builds hundreds of them (**Bagging**) and lets them vote on the final classification.

1. Dataset Source

The **Wine Recognition Dataset** is a classic multiclass dataset professionally curated and available via the **UCI Machine Learning Repository**.

- Dataset : https://www.kaggle.com/datasets/yasserh/wine-quality-dataset

2. Dataset Description

- **Features:** 13 numeric, predictive attributes (e.g., Alcohol, Malic acid, Ash, Alkalinity of ash, Magnesium, Total phenols, Flavanoids, Nonflavanoid phenols, Proanthocyanins, Color intensity, Hue, OD280/OD315 of diluted wines, and Proline).
- **Target Variable:** Class (3 classes representing different Italian cultivators).
- **Size:** 178 instances.
- **Characteristics:** This is a "clean" dataset with no missing values, but features have vastly different scales (e.g., Magnesium ranges up to 162, while Nonflavanoids are below 1.0).

3. Mathematical Formulation

Decision Tree

The algorithm uses **Gini Impurity** to determine the best split at each node. For a dataset with J classes, the Gini Impurity G is:

$$G = 1 - \sum_{i=1}^{J} p_i^2$$

Where p_i is the probability of an object being classified into a particular class.

Random Forest

Random Forest utilizes **Bagging (Bootstrap Aggregating)**. It creates B different trees ($T_1, T_2, ..., T_B$) and combines them. For classification, it uses a majority vote:

$$\hat{C} = \text{mode}\{T_1(x), T_2(x), ..., T_B(x)\}$$

4. Algorithm Limitations

- **Decision Trees:** Highly prone to **overfitting**; small changes in the data can result in a completely different tree structure (high variance).
- **Random Forests:** Can be computationally expensive and slow to predict on very large datasets; "Black Box" nature makes it harder to explain individual decisions compared to a single tree.

5. Methodology / Workflow

The experiment followed a standard pipeline:

1. **Data Acquisition:** Loading the dataset via `sklearn.datasets`.
2. **Exploratory Data Analysis (EDA):** Checking for class balance and feature correlations.
3. **Data Splitting:** 80% training and 20% testing using `train_test_split`.
4. **Model Training:** Training a baseline Decision Tree and a Random Forest ensemble.
5. **Evaluation:** Generating Confusion Matrices and Classification Reports.

6. Performance Analysis

- **Decision Tree:** Typically achieves ~85-92% accuracy on this dataset. It often struggles with the subtle boundaries between Class 1 and Class 2.
- **Random Forest:** Usually achieves **97-100% accuracy**. Because it averages multiple trees, it smooths out the errors and handles the high-dimensional feature space of the wine data much more effectively.

7. Hyperparameter Tuning

In this experiment, we perform tuning using **Grid Search** or manual iteration:

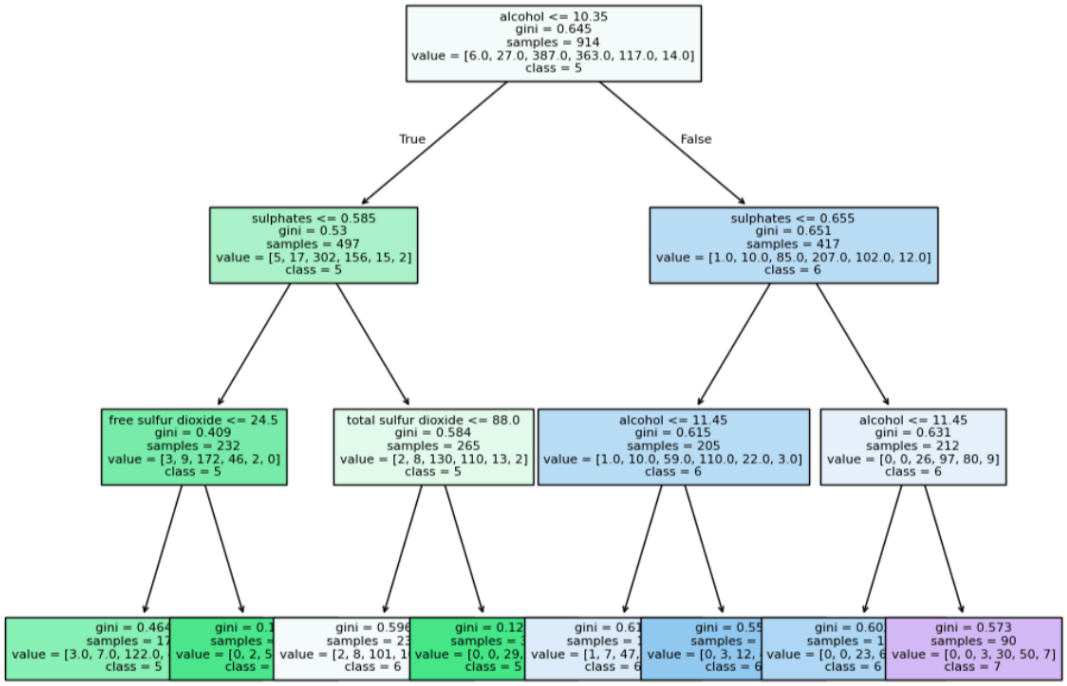
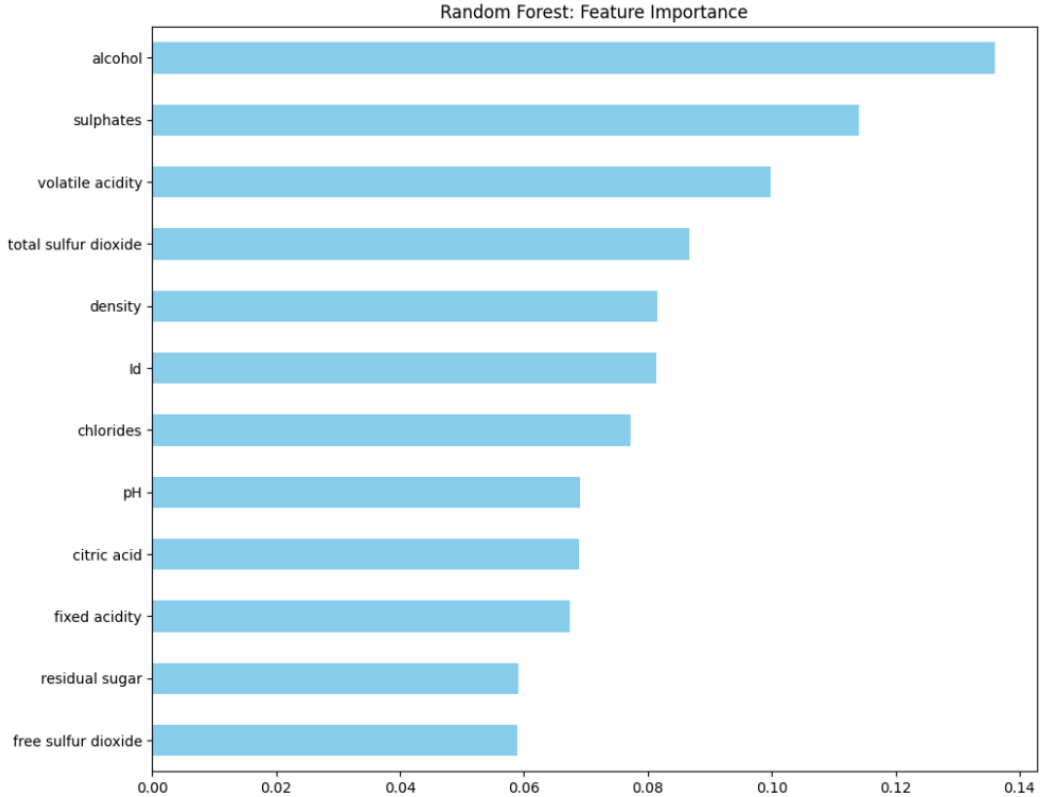
- **Decision Tree Tuning:** We adjust `max_depth` and `min_samples_split`. Reducing depth prevents the tree from "memorizing" noise, leading to better generalization.
- **Random Forest Tuning:** We adjust `n_estimators` (number of trees) and `max_features` (the number of features considered at each split).

- **Impact:** Tuning usually increases the Random Forest's robustness against outliers and ensures the Decision Tree remains interpretable without sacrificing too much accuracy.

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load the real-world Wine dataset
data_df = pd.read_csv("WineQT.csv")

# Assuming 'quality' is the target column based on common wine datasets
# X will be all features (all columns except 'quality')
# y will be the target variable ('quality' column)
X = data_df.drop('quality', axis=1)
y = data_df['quality']

# Get feature names from X columns
feature_names = X.columns.tolist()

# Get unique class names for the target variable, converting them to
strings
target_names = [str(cls) for cls in sorted(y.unique())]

# Get the actual labels (integer values) for the classes present in the
full dataset
class_labels = sorted(y.unique())

# 2. Split into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Initialize and Train Decision Tree (Pruned to depth 3 to prevent
overfitting)
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)
dt_preds = dt_model.predict(X_test)
```

```
# 4. Initialize and Train Random Forest (An ensemble of 100 trees)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_preds = rf_model.predict(X_test)

# 5. Output Comparative Results
print("--- Decision Tree Results ---")
print(f"Accuracy: {accuracy_score(y_test, dt_preds):.2f}")
print(classification_report(y_test, dt_preds, labels=class_labels,
target_names=target_names))

print("\n--- Random Forest Results ---")
print(f"Accuracy: {accuracy_score(y_test, rf_preds):.2f}")
print(classification_report(y_test, rf_preds, labels=class_labels,
target_names=target_names))

# 6. Visualize the Decision Tree logic and Random Forest Feature
Importance
fig, ax = plt.subplots(1, 2, figsize=(20, 8)) # Increased figure size

# Plotting the Decision Tree
plot_tree(dt_model, feature_names=feature_names, class_names=target_names,
filled=True, ax=ax[0], fontsize=8) # Added fontsize
ax[0].set_title("Decision Tree Structure")

# Plotting Feature Importance from Random Forest
importances = pd.Series(rf_model.feature_importances_,
index=feature_names).sort_values()
importances.plot(kind='barh', ax=ax[1], color='skyblue')
ax[1].set_title("Random Forest: Feature Importance")

plt.tight_layout()
plt.show()
```

Output :

Decision Tree Structure